

Why the Sum-of-Exponentials Trick Works

A Proposal: Replacing Fractional-Derivative Memory with a Fixed Number of Exponential States

August 23, 2026

What this document is. A self-contained explanation of the single mathematical idea that makes `fracmem` possible: an exact identity that rewrites a slowly-decaying power law as a continuum of exponentials, discretized into a short, fixed-size sum. No prior background in fractional calculus is assumed — every symbol is defined where it first appears, and every abstract step is followed by real numbers, either hand-computed or produced by actually running the code (`fracmem_pypi/src/fracmem/soe.py`). The goal is to make the case that this technique is worth adopting more broadly, not just describe it.

Contents

1	The problem this solves	1
2	The key idea: exponentials remember cheaply, power laws don't	2
3	The exact identity that makes this possible	2
3.1	The Gamma function, briefly	2
3.2	The identity, derived from scratch	2
4	From an infinite continuum to a short, fixed sum	3
4.1	Step 1: only integrate over the range of s that matters	3
4.2	Step 2: pick a finite number of representative s values, log-spaced	3
5	A tiny example you can trace by hand	4
6	What the real code produces (verified, not estimated)	4
7	The one correction the pure math needs	5
8	Why this pays off: from a memory kernel to a running state	5
9	Why this is worth proposing	6
10	Where this lives in the code	6
11	Further reading	6

1 The problem this solves

A fractional-order derivative of order α (a non-integer, e.g. $\alpha = 0.5$, a “half derivative”) is, in discrete time, a weighted sum over the *entire past* of a signal:

$$D^\alpha x(t_k) \approx h^{-\alpha} \sum_{j=0}^k w_j x_{k-j}, \quad w_0 = 1, \quad w_j = w_{j-1} \left(1 - \frac{1+\alpha}{j}\right).$$

Here k is which sample we want the answer at, j is how many steps back we’re looking, x_{k-j} is the sample j steps before k , h is the time between samples, and w_j is a fixed weight (depends only on α and j , never on the data) for how much that sample j steps back still matters. The values w_j are called the **Grünwald–Letnikov (GL) weights**.

The weights decay, but only as a **power law**:

$$w_j \sim j^{-(1+\alpha)}, \quad j \rightarrow \infty.$$

A power law decays far too slowly to ever safely truncate the sum. Concretely: an ordinary exponential decay ρ^j (say $\rho = 0.9$) has essentially vanished by $j \approx 100$ ($0.9^{100} \approx 2.7 \times 10^{-5}$); a power law $j^{-1.5}$ at $j = 100$ is still 10^{-3} , and at $j = 10,000$ is still 10^{-6} — a thousand times smaller, not gone. Every past sample keeps contributing, forever. That means computing $D^\alpha x(t_k)$ exactly costs more with every new sample ($O(k)$ work, $O(k)$ memory for that one output), and the total cost of a length- n signal is $O(n^2)$.

The one-sentence goal of this whole document: find a way to replace the power-law weight sequence w_j with something that can be tracked using a *fixed* number, updated with a *fixed* amount of work per new sample — no matter how long the signal has been running.

2 The key idea: exponentials remember cheaply, power laws don’t

Consider a single exponential sequence λ^j for some fixed $0 < \lambda < 1$ (e.g. $\lambda = 0.9$). It has a property no power law has: it can be updated with *one multiply*, using only the previous value:

$$\lambda^{j+1} = \lambda \cdot \lambda^j.$$

Check it with real numbers: $\lambda = 0.9$, $j = 3$: $\lambda^3 = 0.729$, and $\lambda^4 = 0.9 \times 0.729 = 0.6561$, which is exactly 0.9^4 . You never need to remember j itself, or recompute from scratch — just keep one running number and multiply by λ each step. This is called the **semigroup property**, and it is the entire reason exponentials are cheap: the whole infinite history compresses into one running state.

A power law has no such shortcut. There is no fixed number ρ with $j^{-\beta} = \rho \cdot (j-1)^{-\beta}$ for every j (the ratio $\left(\frac{j-1}{j}\right)^\beta$ keeps changing as j grows) — so a power-law memory genuinely cannot be compressed into a single running state the way an exponential can.

So the plan is: write the power-law kernel w_j as a combination of a handful of exponentials, $w_j \approx \sum_{i=1}^p c_i \lambda_i^j$. If we can do that honestly, each exponential term becomes one cheap running state, and the whole long-memory sum collapses to p multiply-adds per sample, forever.

3 The exact identity that makes this possible

3.1 The Gamma function, briefly

The Gamma function generalizes the factorial: $\Gamma(n) = (n-1)!$ for a positive integer n (e.g. $\Gamma(1) = 0! = 1$, $\Gamma(4) = 3! = 6$), and for any real $\beta > 0$ it is defined by the integral

$$\Gamma(\beta) = \int_0^\infty x^{\beta-1} e^{-x} dx.$$

That's the only property of Γ this document needs.

3.2 The identity, derived from scratch

Claim: for any $\beta > 0$ and any $j > 0$,

$$j^{-\beta} = \frac{1}{\Gamma(\beta)} \int_0^\infty s^{\beta-1} e^{-sj} ds.$$

Derivation. Start from the definition of $\Gamma(\beta)$ above, and substitute $x = sj$ (so $s = x/j$, $ds = dx/j$):

$$\Gamma(\beta) = \int_0^\infty x^{\beta-1} e^{-x} dx = \int_0^\infty (sj)^{\beta-1} e^{-sj} j ds = j^\beta \int_0^\infty s^{\beta-1} e^{-sj} ds.$$

Divide both sides by j^β :

$$\frac{\Gamma(\beta)}{j^\beta} = \int_0^\infty s^{\beta-1} e^{-sj} ds \quad \implies \quad j^{-\beta} = \frac{1}{\Gamma(\beta)} \int_0^\infty s^{\beta-1} e^{-sj} ds. \quad \blacksquare$$

Read this identity in words: the power law $j^{-\beta}$ (hard to track recursively) is *exactly* equal to a continuous, weighted blend of exponentials e^{-sj} (each one trivially trackable), where s ranges over every possible decay rate from 0 to ∞ , weighted by $s^{\beta-1}/\Gamma(\beta)$. Nothing is approximated yet — this is an exact equality.

For the GL tail, the relevant exponent is $\beta = 1 + \alpha$ (matching $w_j \sim j^{-(1+\alpha)}$ above), and this is exactly the identity used in `fracmem_pypi/src/fracmem/soe.py`:

```
beta = alpha + 1.0
```

4 From an infinite continuum to a short, fixed sum

The identity above still has an infinite continuum of exponentials inside an integral — not yet a finite, computable sum. Two practical steps turn it into one.

4.1 Step 1: only integrate over the range of s that matters

As $s \rightarrow 0$, e^{-sj} decays negligibly slowly (an almost-flat, near-permanent memory); as $s \rightarrow \infty$, e^{-sj} vanishes almost immediately. Neither extreme contributes anything useful past a point, so the code truncates:

```
s_min = 1.0 / max(m_max, 1) # m_max: longest lag we still care about
s_max = 8.0                 # e^-8 = 0.000335: negligible after one sample
```

$m_{\max}$ is the longest distance-into-the-past the design needs to stay accurate for (e.g. 2,000 samples). $s_{\min} = 1/m_{\max}$ is chosen so the slowest exponential included, $e^{-s_{\min} \cdot m_{\max}} = e^{-1} \approx$

0.37, has *just barely* not decayed away by the time you reach the longest lag you asked for — any slower and it would be indistinguishable from “never decays,” which is not representable by a finite sum anyway.

4.2 Step 2: pick a finite number of representative s values, log-spaced

With the range $[s_{\min}, s_{\max}]$ fixed, approximate the integral by a finite sum evaluated at p representative points $s_1, \dots, s_p$ (this is called **quadrature**). The code works in $u = \ln s$ instead of s directly:

```
u_min, u_max = np.log(s_min), np.log(s_max)
du = (u_max - u_min) / p
u = u_min + (np.arange(p) + 0.5) * du # midpoint of each of the p slices
s = np.exp(u)
```

Why log-spacing, concretely: with $m_{\max} = 2000$, the range is $s \in [0.0005, 8]$ — four orders of magnitude. Evenly-spaced points in s itself would put nearly all p points above $s = 1$ and leave the slow, long-memory end (which is exactly the hard part of a power law) almost unrepresented. Evenly-spaced points in $u = \ln s$ instead place points evenly across *scales*: one near $s \approx 0.001$, one near $s \approx 0.01$, one near $s \approx 0.1$, one near $s \approx 1$, and so on. That matches how the power law itself behaves — roughly self-similar across scales — far better than a linear grid could.

The quadrature midpoint rule, $\int f(u) du \approx \Delta u \sum_i f(u_i)$, applied to the identity from Section 3 (after substituting $s = e^u$, so $ds = s du$) gives

$$j^{-\beta} \approx \frac{1}{\Gamma(\beta)} \sum_{i=1}^p \Delta u s_i^\beta e^{-s_i j}.$$

Matching this to the target form $w_j \approx \sum_i c_i \lambda_i^j$ identifies

$$\boxed{\lambda_i = e^{-s_i}} \quad \text{and} \quad \boxed{c_i = \frac{A}{\Gamma(\beta)} \Delta u s_i^\beta e^{-s_i L}}$$

where $A = 1/|\Gamma(-\alpha)|$ is a normalization constant tied to the specific GL tail kernel (not the generic identity), and L is the “local window” length — the tail sum is only ever evaluated for lags $j \geq L$, so the weight is centered there. This is exactly the code:

```
lam = np.exp(-s)
c = (A / gamma(beta)) * du * (s ** beta) * np.exp(-s * L)
```

5 A tiny example you can trace by hand

Take $\alpha = 0.5$ (so $\beta = 1.5$), $L = 32$, $m_{\max} = 1000$, and just $p = 3$ modes — small enough to trace every number:

$$A = \frac{1}{|\Gamma(-0.5)|} = \frac{1}{3.544908} = 0.282095, \quad \Gamma(1.5) = 0.886227,$$

$$s_{\min} = 0.001, \quad s_{\max} = 8, \quad u_{\min} = \ln(0.001) = -6.907755, \quad u_{\max} = \ln 8 = 2.079442,$$

$$\Delta u = \frac{2.079442 - (-6.907755)}{3} = 2.995732.$$

Every one of these numbers comes straight from the four boxed formulas above — there is nothing else going on. (Mode $i = 2$ ’s weight is vanishingly small here only because $L = 32$

i	$u_i = u_{\min} + (i + 0.5)\Delta u$	$s_i = e^{u_i}$	$\lambda_i = e^{-s_i}$	c_i
0	-5.409889	0.004472	0.995538	2.4716×10^{-4}
1	-2.414157	0.089443	0.914441	1.4576×10^{-3}
2	0.581575	1.788854	0.167152	3.1459×10^{-25}

pushes $e^{-s_i L}$ to underflow for a fast-decaying mode; this is expected and harmless, not an error.) With just these three exponentials, the approximation is already

$$w_j \approx c_0 \lambda_0^j + c_1 \lambda_1^j + c_2 \lambda_2^j,$$

a finite, three-term stand-in for a power law that nominally needs the entire past.

6 What the real code produces (verified, not estimated)

Three modes is enough to see the mechanism; it is not enough for real accuracy. The library’s actual default is $p = 16$. Running `soe_tail_kernel(alpha=0.5, L=32, p=16, ...)` for real (not hand-computed) gives 16 modes ranging from a fast-decaying $\lambda_{15} = 0.002708$ down to an extremely slow $\lambda_0 = 0.999324$, and after one additional refinement step (Section 7), the reconstructed tail matches the true GL weights w_j to:

lag j	exact w_j	SOE approximation	relative error
32	-1.576933×10^{-3}	-1.777332×10^{-3}	12.7%
64	-5.542212×10^{-4}	-5.542923×10^{-4}	0.013%
128	-1.953691×10^{-4}	-1.954030×10^{-4}	0.017%
256	-6.897189×10^{-5}	-6.895253×10^{-5}	0.028%
512	-2.436736×10^{-5}	-2.437471×10^{-5}	0.030%
1000	-8.923968×10^{-6}	-8.922978×10^{-6}	0.011%
2000	-3.154507×10^{-6}	-3.152163×10^{-6}	0.074%
4000	-1.115182×10^{-6}	-1.111387×10^{-6}	0.340%

Worst-case relative error across the entire tail: **0.34%**, with 16 numbers standing in for a sum that would otherwise need every one of thousands of past samples. The one weak point is right at $j = L$ (12.7% error, one lag past the boundary) — expected, and addressed directly next.

7 The one correction the pure math needs

The quadrature weights c_i from Section 4 are asymptotically correct (accurate for $j \gg L$), but the true GL kernel isn’t a perfect power law right at the boundary $j = L$ — there’s a small, known mismatch. `soe_refit_weights` corrects exactly this: it keeps every λ_i from the quadrature untouched (they are never re-derived from data), and re-solves for c_i by ordinary linear least squares against the *exact known* GL weights w_j at a few hundred sample lags:

$$\min_c \sum_m \left(\underbrace{\sum_i c_i \lambda_i^m}_{\text{SOE approximation}} - \underbrace{w_{L+m}}_{\text{exact, known}} \right)^2 / |w_{L+m}|^2.$$

This is not machine learning, and it is not fit to any real signal. w_j is a closed-form, exactly-known sequence (Section 1) — the refit is comparing the approximation against *known ground truth*, the same way you’d round π to a nearby fraction and check the error against the true value of π , not against a dataset. This step is what takes the

worst-case error from a few tens of a percent down to the 0.34% figure in Section 6.

8 Why this pays off: from a memory kernel to a running state

This is the actual payoff, and it resolves a subtlety worth being precise about. Once (λ_i, c_i) are fixed, the deployed filter never touches the formulas above again — it uses the semigroup property from Section 2 directly. Define a running state per mode,

$$m_i[k] = \sum_{j \geq 0} \lambda_i^j x_{k-L-j}.$$

Because $\lambda_i^{j+1} = \lambda_i \cdot \lambda_i^j$, this satisfies the one-line recursion

$$m_i[k] = \lambda_i m_i[k-1] + x_{k-L}$$

— exactly the code in `fracmem.py`/src/fracmem/filter.py’s `diagonal_recurrence`: `m = lam * m + x.delayed[k]`. Note $\lambda_i = e^{-s_i}$ (a *per-sample* decay factor, matching this discrete recursion directly) rather than s_i itself (which would be the right quantity only for a *continuous-time* state equation $\dot{m}_i = -s_i m_i + \dot{x}$); the code’s discrete-time use of `lam = np.exp(-s)` is the correct choice for exactly this reason.

The tail estimate is then just $\text{tail}[k] = \sum_i c_i m_i[k]$: p multiply-adds and p stored numbers, updated once per sample, forever — replacing the $O(k)$ -and-growing direct sum from Section 1 with fixed $O(p)$ work and $O(p)$ memory, independent of how long the signal has been running.

9 Why this is worth proposing

- **Complexity.** A signal of length n costs $O(n^2)$ total to differentiate exactly (Section 1); with SOE it costs $O(n \cdot p)$, with p typically 16–32 — a constant, chosen once, independent of n .
- **Memory.** Exact computation needs to remember the entire signal history ($O(n)$, growing forever); SOE needs $O(p)$ numbers, fixed for the life of the device.
- **Certifiable, not just fast.** Because the decay rates λ_i come from an exact identity rather than a data fit, the worst-case error against the true kernel can be computed *before* ever seeing a real signal (Section 6) — this is a property empirical/black-box approximations don’t generally have.
- **Applicability.** Any setting with a fractional-order or long-memory power-law kernel benefits the same way: fractional-order PID control, viscoelastic material models, battery impedance models, and more generally any $t^{-\beta}$ -type memory kernel, fractional or not.

10 Where this lives in the code

11 Further reading

This is a specific instance of a well-established family of techniques in numerical analysis for accelerating power-law/fractional-memory kernels via sum-of-exponentials approximations, including fast evaluation of the Caputo fractional derivative (Jiang et al., <https://arxiv.org/abs/1511.03453>) and more recent exponential-sum approximations for fractional differential equations (<https://arxiv.org/abs/2508.20311>).

File / function	What it does
<code>soe.py : soe.nodes_and_weights</code>	Sections 4–5: pure math, builds $(\lambda_i, c_i^{(0)})$ from the Gamma identity, no data.
<code>soe.py : soe.refit_weights</code>	Section 7: re-solves for c_i against the exact known kernel (still no signal data).
<code>soe.py : soe.tail_kernel</code>	Runs both of the above in sequence; this is what CompressedFractionalFilter calls internally.
<code>soe.py : soe.tail_error</code>	Computes the worst-case relative error number reported in Section 6.
<code>filter.py : diagonal_recurrence</code>	Section 8: the deployed $O(p)$ -per-sample recursion.

Summary, in one paragraph. A power-law memory kernel cannot be tracked with a fixed amount of state — but an exact mathematical identity rewrites it as a continuum of exponentials, which *can*. Discretizing that continuum into a handful of representative exponentials (log-spaced, since the kernel spans many time scales), then correcting the resulting weights against the exact known kernel near the boundary, turns an unbounded-memory power-law sum into a fixed-size, fixed-cost recursive filter — with a computable worst-case accuracy guarantee, before ever touching real data.